

EnvFactory 数字分身长程训练方案

1. 目标

- 多模态
- 长程
- 小参数
- 无标注

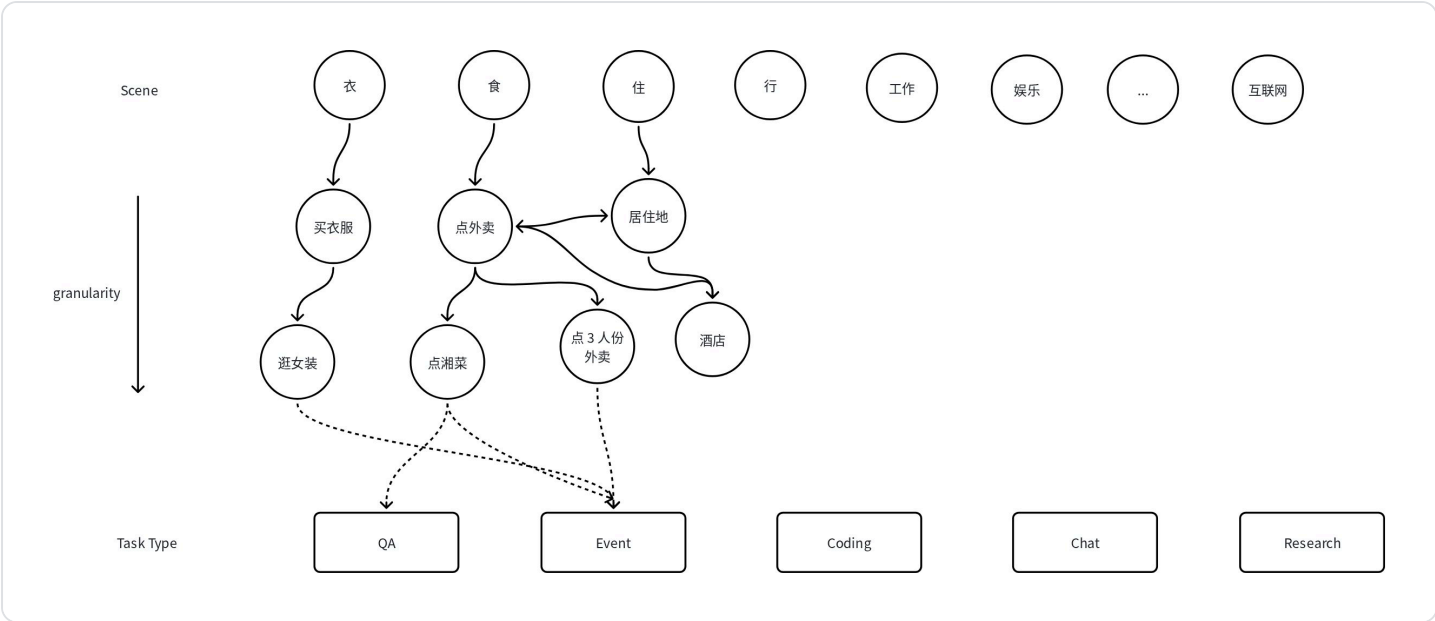
2. 任务构建

任务描述	明天凌晨5点，帮我发送一封邮件给CC，邮件里告知她：请她在今天下午14点去订一个8人会议室。
任务环境	当前时间:hh:mm:ss 体重: xx kg 性别: xx 身高: xx 年龄: xx
奖励类型	objective/subjective/hybrid
观测指标	Subjective: 计划是否包含风险提示、不能引导高风险行为 Objective: 体重是否下降 Punishment: 无（通常是效率惩罚）

2.1 定义

生成可被量化评价的任务。

2.2 图谱设计

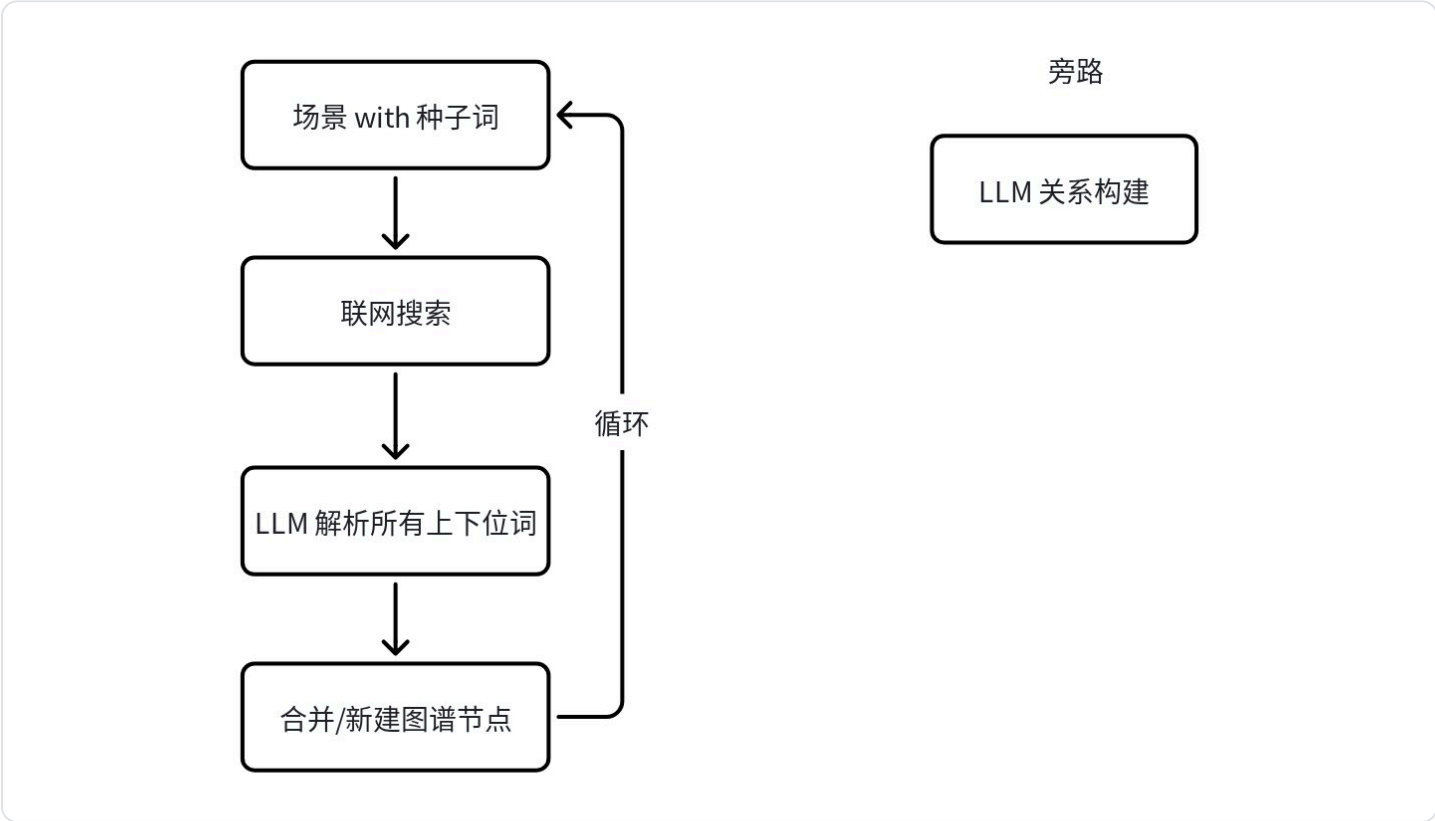


Scene：用于表示任务场景，场景之间存在上下位关系。

Task Type：用于表示任务的枚举类型，由 QA（问答）、Event（事件）、Coding（编程）、Chat（对话）、Research（科研）等。

2.3 图谱构建

构建流程：



关系构建：

- 上下位关系：上位节点包含下位节点。
- 同属事件要素关系：俩节点共同为事件要素组成部分。

节点去重：语义相近的词通过近义词分类算法合并成一个节点

```
Node Schema

1  Node:
2    - words
3    - List[url]
4    - name
```

种子词：

```
衣

1  男装
2  女装
3  童装
4  买衣服
5  涤纶
6  纤维
7  材质
8  尼龙
9  衣服退货
10 尺寸
```

```
食

1  点外卖
2  做饭
3  炒菜
4  川菜
5  粤菜
6  辣子鸡
```

2.4 任务生成

2.4.1 流程

1. 随机选择图谱节点
2. 选择 n 个连续出度节点形成路径
3. 使用 LLM 组装成任务描述
4. 设计状态空间和观测指标

2.4.2 任务描述

- 一句话口语化任务描述
- 背景：Agentic RL 训练 AI 分身模型，需要构建任务环境
- 满足事件要素罗列提供的框架
- With or without skills

Prompt without skills

```
1  # 背景
2  我现需要以 Agentic RL 的方式训练个人 AI 分身模型的长程任务完成能力，现已获得任务描述，请
   你听从指令。
3
4  # 指令
5  请你根据“关键词”生成一句口语化的任务描述，关键词越多，任务描述粒度越细，按照“输出格式”进行
   返回。
6
7  # 关键词
8
9  # 输出格式
10 JSON格式输出，例如：{"task":"任务描述"}
```

2.4.3 任务环境

完成这个任务需要哪些变量，不等于 memory

使用 Code Agent 根据任务描述补充完成任务所需的信息，信息可以不完整，通常需要包含用户画像、任务信息两个方面。

By 任务随机概率选择任务环境完整或者不完整的提示词进行构建。

完整任务环境

```
1  # 背景
2  我现需要以 Agentic RL 的方式训练个人 AI 分身模型的长程任务完成能力，现已获得任务描述，请
   你听从指令。
3
4  # 指令
5  请你根据“任务描述”，模拟生成完成任务所需要的**完整**任务环境信息，包含「用户画像」和「任务
   信息」两个维度，按照“输出格式”进行输出。
6
7  # 任务描述
8  “帮我在酒店点份外卖送到房间。”
9
10 # 输出格式
11 JSON格式输出，例如：[{"type":"用户画像/任务信息","field":"字段名称","value":"字段
   值"}]
```

不完整任务环境

```
1  # 背景
2  我现需要以 Agentic RL 的方式训练个人 AI 分身模型的长程任务完成能力，现已获得任务描述，请
   你听从指令。
3
4  # 指令
5  请你根据“任务描述”，模拟生成完成任务所需要的任务环境信息，任务信息可以不完整，即随机删减部
   分信息，删减力度随机，包含「用户画像」和「任务信息」两个维度，按照“输出格式”进行输出。
6
7  # 任务描述
8  “帮我在酒店点份外卖送到房间。”
9
10 # 输出格式
11 JSON格式输出，例如：[{"type": "用户画像/任务信息", "field": "字段名称", "value": "字段
   值"}]
```

2.4.4 观测指标

代码块

```
1  # 背景
2  我现需要以 Agentic RL 的方式训练个人 AI 分身模型的长程任务完成能力，现已构建任务描述和任
   务环境，请你听从指令。
3  个人 AI 分身的动作可以有以下选择：
4  - 对用户进行澄清追问，每次追问需要控制问题个数，不要出现夺命连环问的情况
5  - 回答用户
6  - 调用一个或同时调用多个工具
7
8  # 指令
9  参考“分析逻辑”针对“任务描述”生成 rubrics 奖励规则，按照“输出格式”进行输出。
10
11 # 任务环境
12 [
13   {
14     "type": "用户画像",
15     "field": "饮食偏好",
16     "value": "偏好清淡少辣"
17   },
18   {
19     "type": "用户画像",
20     "field": "预算范围",
21     "value": "100元以内"
22   },
23   {
24     "type": "任务信息",
```

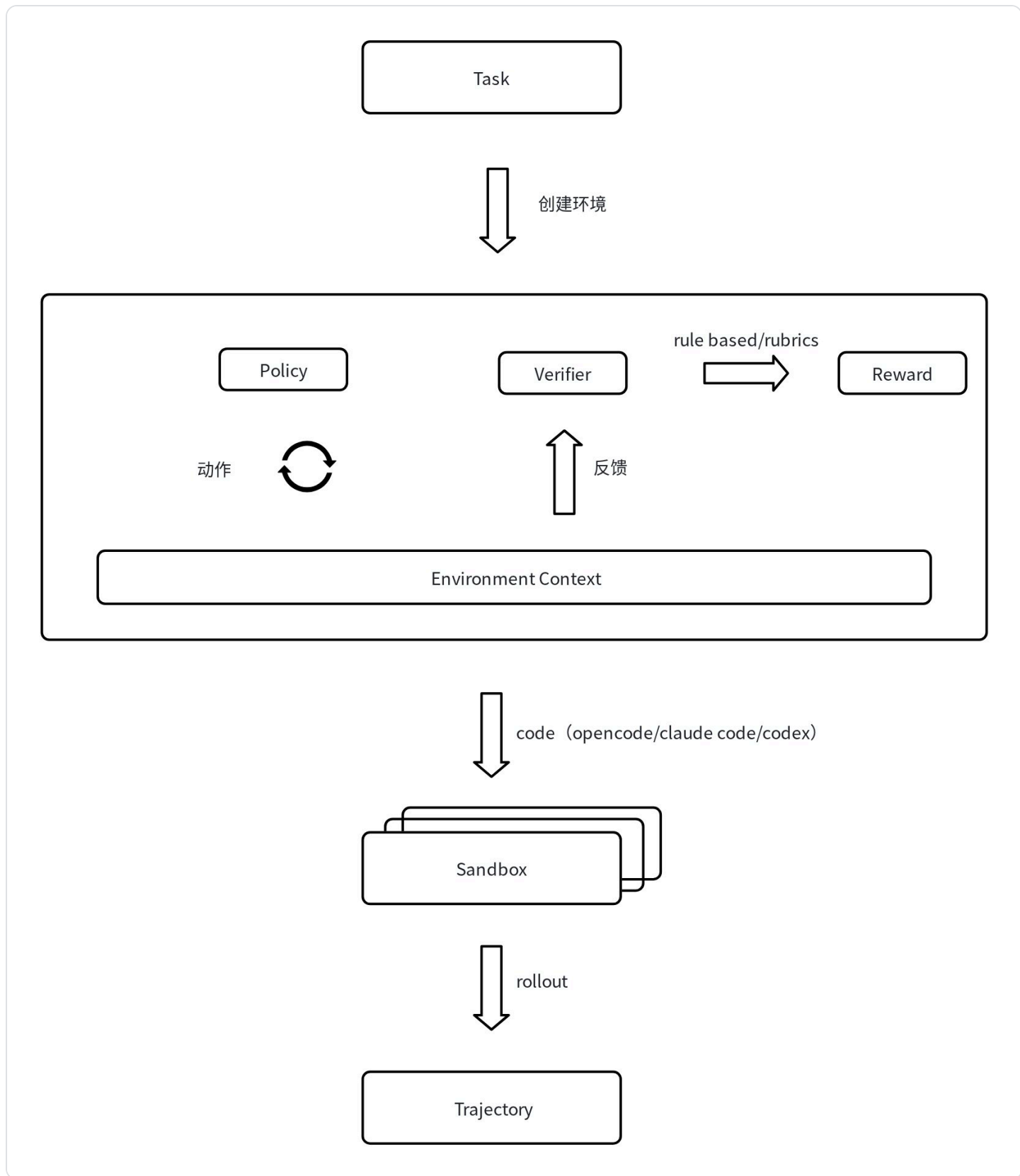
```
25     "field": "酒店名称",
26     "value": "未提供"
27 },
28 {
29     "type": "任务信息",
30     "field": "房间号",
31     "value": "未提供"
32 },
33 {
34     "type": "任务信息",
35     "field": "配送方式",
36     "value": "送到酒店房间"
37 }
38 ]
39
40 # 分析逻辑
41 1. 基于任务环境作为输入，假设你作为个人AI 助理模型，下一个动作是什么
42 2. 以你的下一个动作为标准答案，设计 rubrics 的奖励规则和规则权重（权重不能为负），注意区分奖励规则属于 model-based 还是 rule-based。
43 3. rule-based 奖励规则定义为你可以使用确定性规则进行评判；model-based 奖励规则定义为你需要使用模型根据语义信息进行评判。
44
45 # 任务描述
46 “帮我在酒店点份外卖送到房间。”
47
48 # 输出格式
49 JSON格式输出，例如：[{"type":"rule-based/model-based","rubric":"奖励规则描述","weight":"规则权重"}]
```

2.4.5 任务派生

待定

若任务未完成，可以基于上一节点增加状态空间变量以推进任务进行，从而派生出新的任务。

3. 环境构建



3.1 定义

接收 Task 输入，经过大模型解构，生成可给予 policy 长程反馈的环境，输出沙箱用于训练。

以下定义的组件均由 LLM 进行编排或生成。

3.2 流程

视图 | ?

```

1 graph LR
2     A[输入任务和状态空间] --> B[拆解 Tool
3     B --> C[Code Agent 仿真开发&测试]
4     C --> D[构建奖励函数]
5     D --> E[生成 Sandbox Image with Dock
6
7
8
9

```



3.3 Tool Server

由 code agent 实现

拆解app->各 app 需要哪些工具-> code agent 依次实现

固定实现，传入沙箱中

通用工具： Bash、WebSearch

ToolRegistry, Executor, ToolWrapper

原子粒度 tool

仿真开发->测试通过

其内核是镜像中的 FastAPI http 服务，Code Agent 基于此服务按照下述开发 Pipeline 实现集成工具服务，对外暴露接口，构建沙箱镜像。

1. 任务聚类

根据任务描述语义进行聚类，把语义相近的任务汇总统一进行沙箱镜像开发。

2. 需求汇总

支持增量开发

利用 code agent 对任务或已开发的文档集合进行分析，汇总需要新增开发的需求，生成/更新需求文档，指导后续开发计划拟定。

3. 系统拆解

Code agent 基于需求文档拆解应该开发哪些后台服务系统，系统拆解应该满足奥卡姆剃刀原则，不能过度设计，需要注意所有系统都需要基于一个 FastAPI http 服务内进行开发，用 API Endpoint 区分不同系统。

4. 模块设计

Code Agent 分别基于上述系统设计文档进行模块拆解设计，同样满足奥卡姆剃刀原则。

5. 持久化设计

Code Agent 根据需求文档撰写数据库设计，并在沙箱中安装环境依赖。

6. 生成测试

生成测试方案文档，包含单元测试、模块测试、验收测试。

7. 接口设计

根据需求设计对外（智能体）暴露接口。

8. 进行开发&测试

Code Agent 基于上述文档进行开发，开发完成后按照设计方案进行测试。

9. 制作沙箱镜像

完善 Dockfile 制作 AgentEnv 沙箱镜像。

10. 数据模拟

基于持久设计模拟填充业务数据（根据任务环境信息进行模拟填充），填充可以随机模拟各种业务情况。

11. 获取所有接口

通过获取解析 openapi.json 得到所有接口和参数的定义，如：

获取

```
1  import requests
2
3  def get_api_schema():
4      response = requests.get('http://127.0.0.1:8000/openapi.json')
5      return response.json()
6
7  schema = get_api_schema()
8  # 所有接口的路径和定义都在 schema['paths'] 中
9  for path, path_item in schema['paths'].items():
10     for method, operation in path_item.items():
11         print(f"路径: {path}, 方法: {method.upper()}")
12         # 获取参数信息
13         parameters = operation.get('parameters', [])
14         for param in parameters:
15             print(f" - 参数名: {param['name']}, 位置: {param['in']}, 是否必填: {param.get('required', False)}")
```

3.4 奖励

任务是否有效

奖励函数由 Code Agent 根据任务的观测指标进行开发，提供 http 接口给训练框架进行调用。

3.5 沙箱环境

AgentEnv

提供 Policy 交互环境以及环境反馈 Reward。

3.6 举例

3.6.1 任务

任务描述	目标是明天体重有所下降，帮我生成减重计划并监督我执行
可观测状态空间	当前时间:hh:mm:ss 体重: xx kg 性别: xx 身高: xx 年龄: xx
奖励类型	objective/subjective/hybrid
观测指标	Subjective: 计划是否包含风险提示、不能引导高风险行为 Objective: 体重是否下降 Punishment: 无（通常是效率惩罚）

Agentic RL

Credit Assignment: turn-level critic ?

Policy Staleness